

Laporan Praktikum Kontrol Cerdas Week 4

Nama : Wahyu Ramadhan A
NIM : 224308047
Kelas : TKA-6B
Akun Github : <https://github.com/wahyuuu12>
Student Lab Assistant : Mas Dimas

1. Judul Percobaan: *Reinforcement Learning for Autonomous Control*
2. Tujuan Percobaan:
 - Memahami konsep dasar *Reinforcement Learning* (RL) dalam sistem kendali.
 - Mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
 - Menggunakan OpenAI *Gym* sebagai simulasi lingkungan untuk pelatihan RL.
 - Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
 - Menggunakan GitHub untuk *version control* dan dokumentasi praktikum.

3. Landasan Teori:

Reinforcement learning adalah belajar apa yang akan dilakukan pembelajaran dengan pemetaan situasi dalam menentukan Tindakan dan memaksimalkan angka sinyal penghargaan yang bisa diperoleh dari lingkungannya (Nguyen, et al., 2018). Tujuan *Reinforcement learning* yaitu menggunakan pengamatan yang dikumpulkan dari interaksi dengan lingkungan untuk mengambil tindakan yang akan memaksimalkan hasil dan meminimalkan risiko (Wijoyo , et al., 2024). Algoritma ini akan terus belajar secara iteratif. Pada algoritma ini terdapat agen yang akan belajar dari interaksi dengan lingkungannya. Untuk menghasilkan sebuah model, penguatan algoritma pembelajaran melalui beberapa tahapan, diantaranya agen mengamati data masukan, setelah itu agen mengambil tindakan untuk

mengambil keputusan. Setelah keputusan diambil, agen akan mendapatkan “reward” atau penguatan dari lingkungan. Kemudian mengamati input kembali dan proses pengambilan keputusan dilakukan kembali namun dengan tambahan penguatan dari lingkungan agar hasil keputusan yang diambil lebih akurat. Algoritma RL salah satunya DQN yang menggabungkan algoritma Q-Learning dengan Deep Neural Network (Fuad, 2022). Deep Q-Network (DQN) adalah algoritma dalam *Reinforcement Learning* (RL) yang menggunakan jaringan saraf dalam (*deep neural network*) untuk mendekati fungsi Q, yaitu fungsi yang memetakan keadaan (*state*) dan tindakan (*action*) ke nilai yang mengukur seberapa baik tindakan tersebut dalam jangka panjang. Q-Learning dikategorikan dalam metode model-free yang tidak membutuhkan informasi yang lengkap tentang kondisi Environment. Beberapa jenis environment yang digunakan dalam RL diantaranya LunarLander-v2, MountainCar-v0 dan cartpole.

4. Analisis dan Diskusi:

- Analisis

Pada percobaan yang telah dilakukan, diterapkan algoritma Deep Q-Network (DQN) dengan tujuan melatih agen dalam dua lingkungan yang berbeda, yaitu **MountainCar-v0** dan **CartPole-v1**. Pada **MountainCar-v0** Agen harus menggerakkan mobil yang terjebak di lembah dengan menggunakan momentum untuk mencapai puncak. Tugas ini cukup menantang karena mobil tidak cukup kuat untuk mendaki secara langsung, sehingga agen harus belajar untuk mengayunkan mobil maju-mundur untuk mencapai tujuannya. Reward diatur berdasarkan kondisi lingkungan. Jika agen berhasil mencapai tujuan (mobil mencapai puncak), agen mendapatkan reward sebesar 100. Jika gagal (waktu habis), agen mendapatkan reward negatif. Agen dibekali dengan jaringan saraf sederhana yang memiliki dua lapisan tersembunyi dan mengadopsi strategi epsilon-greedy untuk eksplorasi. Untuk $\epsilon_{\min} = 0.01$ sedangkan $\epsilon_{\text{decay}} = 0.995$. Parameter seperti gamma (faktor diskon), epsilon (rate eksplorasi), dan ϵ_{decay} (penurunan epsilon) digunakan untuk

pengambilan keputusan dan pembelajaran. Sedangkan pada **CartPole-v1** agen harus menjaga keseimbangan tiang di atas gerobak selama mungkin dengan menggerakkan gerobak ke kiri dan kanan untuk menghindari tiang jatuh. Agen mendapatkan reward untuk setiap langkah di mana tiang tetap seimbang, dan episode berakhir jika tiang jatuh atau batas waktu tercapai. Reward diatur berdasarkan kondisi lingkungan. Agen mendapatkan reward positif jika berhasil menjaga keseimbangan dan reward negatif jika tiang jatuh atau waktu habis. Dengan *epsilon decay* yang relatif cepat, agen lebih cepat beralih dari **eksplorasi** ke **eksploitasi**, yang mempercepat konsistensi pengambilan keputusan berdasarkan pengalaman sebelumnya.. Parameter seperti gamma (faktor diskon), epsilon (rate eksplorasi), dan epsilon_decay (penurunan epsilon) digunakan untuk pengambilan keputusan dan pembelajaran. batas maksimal episode 1000 membuat agen memiliki banyak kesempatan untuk belajar dan menyempurnakan kebijakan yang lebih optimal.

- Diskusi

Dari praktikum yang telah dilakukan dapat diketahui perbedaan hasil pembelajaran antara CartPole-v1 dan MountainCar-v0 menunjukkan bagaimana karakteristik lingkungan yang berbeda mempengaruhi cara agen belajar. Pada CartPole-v1, reward diberikan secara positif untuk setiap langkah yang sukses, agen mendapat reward secara kontinu, yang memberikan feedback langsung untuk memperbaiki kebijakan aksi. Hal ini mempercepat pelatihan dan memudahkan algoritma menemukan strategi yang baik. Ini membuat *CartPole* lebih mudah dipelajari dengan pendekatan eksplorasi yang lebih cepat dan replay buffer yang lebih kecil, karena agen mendapatkan pengalaman positif dengan lebih cepat. CartPole relatif sederhana sehingga cocok untuk eksperimen awal dalam reinforcement learning dan untuk menguji algoritma dasar seperti Q-learning atau DQN. Sebaliknya, di MountainCar-v0, reward hanya signifikan ketika agen mencapai puncak, sehingga agen harus mengembangkan strategi jangka panjang, seperti mengumpulkan

momentum dengan aksi berulang . Reward yang diterima hanya -1 hingga mencapai tujuan, sehingga agen tidak mendapat feedback positif hingga mencapai puncak, membuat proses pelatihan bisa lebih lambat. Membutuhkan lebih banyak eksplorasi dan strategi yang lebih cerdas, sehingga bagus untuk pengujian algoritma DQN dan varian lanjutan. Hal ini membuat MountainCar lebih sulit dipelajari dan membutuhkan *replay buffer* yang lebih besar serta strategi pelatihan yang lebih sabar. Agen memerlukan lebih banyak data untuk memahami bagaimana mencapai tujuan, sehingga teknik seperti Target Network Update dan epsilon decay yang lebih lambat menjadi lebih efektif.

5. *Assigment:*

Pada praktikum minggu keempat ini, telah percobaan dan perbandingan performa agen Deep Q-Network (DQN) pada dua lingkungan berbeda, yaitu CartPole-v1 dan MountainCar-v0 menggunakan pustaka OpenAI Gym Gym, OpenCV dan TensorFlow. Agen DQN yang digunakan memiliki dua neural network utama, yaitu Q-Network untuk memperkirakan nilai Q dan menentukan aksi optimal berdasarkan kondisi lingkungan dan Target Network yang diperbarui secara berkala untuk stabilisasi pelatihan dengan memperbarui bobot secara berkala.

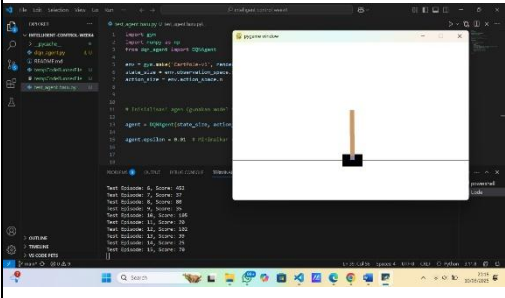
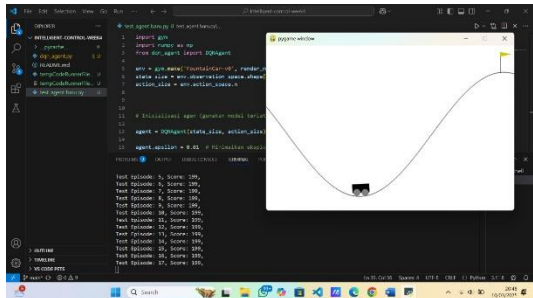
Pada eksperimen pertama dengan CartPole-v1, agen bertugas menjaga keseimbangan tiang di atas gerobak/kotak selama mungkin dengan memperoleh reward +1 setiap langkah yang berhasil dilalui. Reward yang sering memungkinkan pembelajaran yang lebih cepat. Parameter seperti epsilon decay (0.995), replay buffer (2000), dan batch size (64) dan pembaruan target network setiap 10. Hasil eksperimen menunjukkan bahwa agen pada awalnya banyak melakukan eksplorasi karena nilai epsilon yang tinggi (1.0), tetapi seiring berjalannya waktu dan penurunan epsilon, agen mulai mengeksploitasi kebijakan terbaiknya. Dengan semakin banyaknya pengalaman yang dikumpulkan, agen berhasil meningkatkan skor dan menjaga keseimbangan tiang lebih lama. Mekanisme pembaruan Target Network secara

berkala juga berperan dalam menjaga stabilitas pelatihan, sehingga nilai Q tidak berubah terlalu drastis.

Sedangkan, eksperimen kedua dilakukan pada MountainCar-v0, yang memiliki tantangan lebih besar karena reward hanya diberikan jika agen berhasil mencapai puncak bukit. Tantangan utama di lingkungan ini adalah agen harus menemukan strategi yang tepat untuk memanfaatkan momentum guna mencapai tujuan. Untuk mengakomodasi tantangan tersebut, dengan mengubah epsilon decay yang lebih lambat, agen memiliki lebih banyak kesempatan untuk mencoba berbagai strategi sebelum mulai mengeksploitasi. Replay buffer yang diperbesar menjadi 5000, serta penambahan reward +100 saat mencapai puncak untuk memberi sinyal yang lebih jelas kepada agen tentang tujuan akhirnya. Pada awal eksperimen, agen mengalami kesulitan karena tidak ada reward yang langsung diberikan untuk setiap langkah, sehingga strategi eksplorasi menjadi lebih sulit. Namun, dengan semakin banyaknya pengalaman yang dikumpulkan, agen mulai memahami bahwa mengayunkan mobil maju-mundur dapat membantunya mencapai puncak lebih efisien. Setelah beberapa ratus episode, agen secara bertahap mulai lebih sering mencapai tujuan dan menyelesaikan tugas dengan lebih cepat.

6. Data dan Output Hasil Pengamatan:

No.	Variabel	Cartpole-v1	MountainCar-v0
1.	Waktu Pelatihan	CartPole lebih cepat mencapai performa optimal	MountainCar membutuhkan lebih banyak waktu untuk mencapai keberhasilan yang stabil
2.	Jumlah Episode Maksimum	CartPole dilatih hingga 1000 episode <pre> _name__ == '__main__': env = gym.make('CartPole-v1') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 64 target_update_interval = 10 # Interval to u </pre>	MountainCar dilatih hingga 1000 episode <pre> _name__ == '__main__': env = gym.make('MountainCar-v0') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 64 target_update_interval = 10 # Interval to </pre>
3.	Skor dari tiap episode	Skor bervariasi dari beberapa episode <pre> Test Episode: 3, Score: 83 Test Episode: 4, Score: 48 Test Episode: 5, Score: 55 Test Episode: 6, Score: 452 Test Episode: 7, Score: 37 Test Episode: 8, Score: 80 Test Episode: 9, Score: 35 Test Episode: 10, Score: 105 Test Episode: 11, Score: 20 Test Episode: 12, Score: 102 Test Episode: 13, Score: 39 </pre>	Skor awal mencapai 199 karena maksimal langkah dalam satu episode adalah 200 <pre> Test Episode: 2, Score: 199, Test Episode: 3, Score: 199, Test Episode: 4, Score: 199, Test Episode: 5, Score: 199, Test Episode: 6, Score: 199, Test Episode: 7, Score: 199, Test Episode: 8, Score: 199, Test Episode: 9, Score: 199, Test Episode: 10, Score: 199, Test Episode: 11, Score: 199, Test Episode: 12, Score: 199, Test Episode: 13, Score: 199, Test Episode: 14, Score: 199, </pre>

4.	Epsilon	Nilai epsilon (ϵ) menurun seiring dengan proses agen <pre> Epsilon: 1.00 Epsilon: 1.00 Epsilon: 0.99 Epsilon: 0.99 Epsilon: 0.99 Epsilon: 0.98 Epsilon: 0.98 , Epsilon: 0.97 , Epsilon: 0.97 , Epsilon: 0.96 , Epsilon: 0.96 , Epsilon: 0.95 , Epsilon: 0.95 , Epsilon: 0.94 , Epsilon: 0.94 , Epsilon: 0.93 , Epsilon: 0.93 Epsilon: 0.92 , Epsilon: 0.92 , Epsilon: 0.91 , Epsilon: 0.91 , Epsilon: 0.90 , Epsilon: 0.90 </pre>	Nilai epsilon (ϵ) menurun seiring dengan proses agen <pre> Epsilon: 0.95 Epsilon: 0.95 Epsilon: 0.94 Epsilon: 0.94 Epsilon: 0.93 Epsilon: 0.93 Epsilon: 0.92 Epsilon: 0.92 Epsilon: 0.91 Epsilon: 0.91 Epsilon: 0.90 Epsilon: 0.90 Epsilon: 0.90 Epsilon: 0.89 </pre>
5.	Kinerja Agen	Kinerja yang lebih stabil dengan skor yang lebih tinggi	Masih menghadapi tantangan dalam mencapai puncak
6.	Kondisi Berhasil	Episode berakhir jika tiang jatuh, reward akan positif	Episode berakhir jika mobil mencapai puncak bukit (flag)
7.	Hasil		

7. Kesimpulan:

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- MountainCar-v0 lebih sulit dan cocok untuk menguji algoritma reinforcement learning yang lebih canggih karena membutuhkan strategi jangka panjang. Namun, proses pelatihannya bisa lambat karena reward yang tidak instan.
- CartPole-v1 lebih sederhana dan cepat dipelajari, cocok untuk eksperimen awal dalam pengembangan algoritma pembelajaran penguatan, tetapi tidak memberikan tantangan besar untuk algoritma yang lebih kompleks.
- Agen menggunakan pendekatan DQN dengan replay buffer untuk memori, fungsi nilai Q di-*approximate* oleh jaringan saraf (*neural network*), dan *epsilon-greedy* untuk eksplorasi.
- Agen memulai dengan nilai epsilon tinggi (1.0) dan secara bertahap menurunkannya, memungkinkan agen untuk lebih banyak melakukan eksploitasi seiring dengan waktu.

8. Saran:

Untuk meningkatkan agar performa agen DQN lebih baik bisa menerapkan *Target Network* untuk meningkatkan stabilitas dan mencegah model dari *overfitting* terhadap pola jangka pendek, dengan cara memperbarui target *network* secara periodik. strategi epsilon *decay* yang lebih adaptif, seperti menyesuaikannya berdasarkan performa, dapat memperbaiki keseimbangan eksplorasi dan eksploitasi. dapat mengoptimalkan *hyperparameter* seperti gamma, learning_rate, dan epsilon_decay untuk meningkatkan performa. Menggunakan Double DQN akan membantu menghindari masalah overestimation. MountainCar, pendekatan lain seperti *actor-critic methods* atau *policy-based reinforcement learning* dapat dicoba untuk meningkatkan efisiensi pembelajaran.

9. Daftar Pustaka

- Cahya, F., Riana, D. & Hadiyanti, S., 2021. Klasifikasi Penyakit Mata Menggunakan Convolutional Neural Network (CNN). *SISTEMASI*, p. 618.
- Fuad, M. R. T., 2022. Adaptive Deep Q-Network Algorithm with Exponential Reward Mechanisme for Traffic Control in Urban Intersection Networks. *Sustain*, Volume 14, p. 21.
- Malik, M., 2021. DETEKSI SUHU TUBUH DAN MASKER WAJAH DENGAN MLX90614, OPENCV, KERAS/TENSORFLOW, DAN DEEP LEARNING.. *Jurnal Engine: Energi, Manufaktur, dan Material*, p. 19.
- Nguyen, N. D., Nguyen, T. & Nahavandi, S., 2018. System Design Perspective for Human-Level Agent Using Deep Reinforcement Learning: A Survey. *IEEE Acces* , Volume 1, p. 99.
- Pumsirirat, 2018. Credit Card Fraud Detection using Deep Learning based on Auto-Encoder International Journal of Advanced. *ComputerScience and Applications (IJACSA)*, Volume 9(1), pp. 1825-1844.
- Rahman, C. R. et al., 2020. Identification and recognition of rice diseases and pests using convolutional neural networks.. *Biosystems Engineering*, pp. 112-120.
- Santoso, A. & Ariyanto, G., 2018. IMPLEMENTASI DEEP LEARNING BERBASIS KERAS UNTUK PENGENALAN WAJAH. *Jurnal Tekning Elektro*.
- Wijoyo , A. et al., 2024. Pembelajaran Machine Learning. *Jurnal Ilmu Komputer dan Science*, Volume 3, p. 2.